Technical Blueprint for an Ethiopian Financial Intelligence Infrastructure: Architectural Design and Implementation Strategy

1. Executive Strategic Overview

The establishment of the Ethiopian Securities Exchange (ESX) marks a pivotal moment in the economic history of East Africa, signaling a definitive shift from a state-led developmental model toward a market-oriented financial system. As Ethiopia prepares to operationalize its first modern stock exchange, the demand for high-fidelity, real-time market data has shifted from a luxury to an operational necessity. For institutional investors, algorithmic traders, and policy analysts, the ability to aggregate, filter, and act upon disparate information streams—ranging from treasury bill auction results in local broadsheets to macroeconomic policy shifts reported by global wire services—will constitute a primary competitive advantage.

The objective of this technical report is to define the architecture, engineering principles, and implementation roadmap for a specialized Financial News Aggregation System. This system is designed to harvest unstructured data from three primary sources—*Addis Fortune*, *Capital Ethiopia*, and *Reuters*—and transform it into structured, queryable intelligence stored in a local MongoDB database and exposed via a high-performance FastAPI endpoint.

In the context of the Ethiopian digital ecosystem, this task transcends simple web scraping. It requires navigating a complex landscape of technical variability, from the static, WordPress-based architecture of local news portals to the sophisticated, anti-bot-protected infrastructure of international agencies like Reuters. Furthermore, the system must operate within the strict legal boundaries of the Ethiopian Computer Crime Proclamation No. 958/2016, ensuring that data harvesting practices respect digital sovereignty and data integrity mandates.[1]

The proposed solution leverages a "Clean Architecture" approach, utilizing Python's advanced asynchronous capabilities. By integrating `Playwright` for dynamic content rendering, `HTTPX` for high-concurrency static requests, and `FlashText` for algorithmic keyword extraction, the system addresses the specific user requirement to filter for high-value financial keywords such as "treasury bills," "capital markets," and "macroeconomics." The persistence layer, built on MongoDB and the Beanie Object Document Mapper (ODM), ensures schema flexibility to accommodate the erratic nature of web data, while the API layer is designed for scalability and ease of integration.

This document serves as a comprehensive engineering manual, dissecting every component of the pipeline. It provides deep theoretical justifications for library selections, detailed analyses of

target site structures, and rigorous code-level strategies for bypassing anti-scraping defenses, all while maintaining an unwavering focus on the specific financial lexicon requested by the stakeholder.

---

2. Target Source Reconnaissance and DOM Analysis

The foundation of any robust web scraping infrastructure is a granular understanding of the target environments. The three sources identified—*Addis Fortune*, *Capital Ethiopia*, and *Reuters*—present distinct technical challenges that dictate the selection of ingestion strategies. A "one-size-fits-all" scraper would be destined for failure; instead, we must engineer bespoke adapters for each source that respect their unique Document Object Model (DOM) structures, network behaviors, and content delivery networks (CDNs).

2.1 Addis Fortune: The Domestic Financial Anchor

Addis Fortune serves as the definitive record for Ethiopia's private sector, offering critical data on banking regulations, exchange rates, and insurance sector performance. Its importance lies in the "Money Market Watch" and editorial analysis, which often contain the raw data needed for "macroeconomics" and "economics" keyword filtering.

2.1.1 Structural and Network Analysis

An analysis of the *Addis Fortune* digital footprint reveals a WordPress-based architecture serving largely static HTML content.[3] Unlike modern Single Page Applications (SPAs) that rely heavily on client-side JavaScript to render content, *Addis Fortune* delivers the core article text within the initial HTTP response. This allows for the use of lightweight HTTP clients rather than resource-intensive browser automation tools.

The DOM structure of the news feed is categorized into distinct semantic sections. The "Fortune News" and "Money Market Watch" sections are the primary targets. Inspection of the HTML reveals that individual articles are typically encapsulated within container `div` elements, often utilizing Bootstrap-style grid classes (e.g., `col-md-4`). Headlines are universally wrapped in header tags, specifically `<h4>` or `<h5>`, often preceded by distinct markers such as `####` in their markdown representations or visual rendering.[3]

Crucially, the date metadata—essential for time-series analysis of financial trends—is positioned immediately below the headline and preceding the "Read more >>" anchor tags. The format of these dates is non-standard, often appearing as "Dec 6 , 2025" with irregular spacing.[3] This necessitates a robust normalization layer in the data pipeline to convert these string representations into ISO 8601 compliant datetime objects. The summary text, which provides the initial signal for keyword relevance, is contained in a paragraph block siblings to the date element.

2.1.2 RSS and Discovery Limitations

While *Addis Fortune* theoretically maintains an RSS feed (addisfortune.news/feed), reliability is a significant concern in the region's digital infrastructure.[4] Feeds often lag behind the live site or provide only truncated snippets that lack the full context required for deep keyword analysis. Therefore, the scraper architecture cannot rely solely on RSS for "exhaustive" detail. A hybrid discovery strategy is required: the system acts as a crawler, systematically visiting the "Archives" or "Category" pages to ensure no critical "Treasury Bill" auction notice is missed due to a malformed RSS XML item.

2.2 Capital Ethiopia: The Tender and Policy Specialist

Capital Ethiopia complements *Fortune* by focusing heavily on infrastructure, tenders, and granular economic policy. For a user interested in "politics" and "treasury bills," *Capital* is often the primary source for government auction notices and ministry-level announcements.

2.2.1 Pagination and Traversal Logic

The site (capitalethiopia.com) follows a traditional blog structure with pagination implemented via URL path parameters (e.g., /page/2/, /page/3/).[5] This predictable structure simplifies the crawler's traversal logic. The scraper can instantiate a queue of URLs to visit, incrementing the page number until a 404 error or a date cutoff is reached.

The content on *Capital* is particularly rich in "Request for Quotation" (RFQ) and tender notices. These are structurally distinct from standard news articles. The scraper must implement heuristic logic to distinguish between a news article about the economy and a raw tender document. The user's request for "capital markets info" implies a need to filter out purely administrative tenders (e.g., "Supply of School Desks") while retaining economically significant ones (e.g., "Consultancy for Stock Exchange Implementation"). This requires the NLP layer to be context-aware, a feature we will discuss in the FlashText implementation section.

2.2.2 Content Extraction Challenges

Capital frequently embeds crucial data—such as bond yield tables or exchange rate adjustments—within the text body or, occasionally, as images. While Optical Character Recognition (OCR) is outside the immediate scope, the scraper must be resilient to poorly formatted HTML tables. The text extraction logic must preserve the proximity of numerical data to keywords like "Bill" or "Bond" to ensure the stored data retains its semantic meaning.

2.3 Reuters: The International Fortress

The inclusion of *Reuters* introduces a significant jump in technical complexity. As a global tier-one news agency, *Reuters* employs enterprise-grade security to protect its intellectual property. The specific URL provided (reuters.com/.../ethiopia-launches-stock-exchange...) indicates the user's need for high-level international perspective on Ethiopian markets.[6]

2.3.1 Cloudflare and Anti-Bot Defenses

Reuters sits behind Cloudflare's edge network, which utilizes advanced bot detection techniques. These include TLS Fingerprinting (analyzing the cryptographic handshake to identify non-browser clients), behavioral analysis (tracking mouse movements and scroll patterns), and JavaScript challenges (Cloudflare Turnstile).[7]

A standard Python `requests` library will fail to scrape *Reuters*, receiving a `403 Forbidden` response or being redirected to a "Just a moment..." challenge page. The detection mechanisms specifically look for the absence of standard browser headers or the inability to execute complex JavaScript. Furthermore, *Reuters* has deprecated its public RSS feeds, meaning the scraper cannot rely on XML streams for discovery.[9] It must scrape the "World/Africa" or "Markets" section pages directly, rendering them as a human user would.

2.3.2 Dynamic Content Rendering

Unlike the local Ethiopian sites, *Reuters* utilizes a modern frontend framework (likely React or Next.js) where content is hydrated dynamically. The initial HTML response often contains only a skeletal structure. The actual article text is loaded asynchronously via JSON APIs or injected into the DOM via JavaScript. This mandates the use of a "Headless Browser" approach. We will utilize **Playwright**, a modern automation library that can control a real Chromium instance, executing the necessary JavaScript to render the page fully before extraction occurs.[7]

2.4 Comparative Source Matrix

Feature	Addis Fortune	Capital Ethiopia	Reuters
Tech Stack	WordPress (Static HTML)	WordPress (Static HTML)	React/Next.js (Dynamic)
Anti-Bot	Minimal (Standard User-Agent)	Minimal (Standard User-Agent)	**High (Cloudflare)**
Discovery	Archive/Category Pagination	URL Pagination (/page/n)	Section Scraping / Search

Data Richness	High (Money Market)	High (Tenders/Policy)	High (Global Context)
Scraping Tool	HTTPX + BeautifulSoup	HTTPX + BeautifulSoup	**Playwright + Stealth**
Response Time	Fast (<1s)	Fast (<1s)	Slow (>5s per page)

---

3. Legal Framework and Ethical Compliance

The architecture of this system must be underpinned by a solid understanding of the legal landscape. In Ethiopia, the digital domain is governed by strict laws designed to protect national security, critical infrastructure, and data integrity. Ignoring these statutes puts the operator at risk of criminal liability.

3.1 The Computer Crime Proclamation (No. 958/2016)

The primary legislative instrument governing this activity is the **Computer Crime Proclamation No. 958/2016**.[2]

3.1.1 Unauthorized Access (Article 3)

Article 3 criminalizes access to computer systems without authorization. While public news websites are implicitly authorized for human consumption, high-frequency automated access can be construed as an attack or an unauthorized usage of resources. To mitigate this legal risk, the scraper must operate at a "human" cadence. We will implement strict **Rate Limiting** (e.g., one request every 5-10 seconds per domain) to ensure that the scraping activity does not degrade the service of the target websites, which would arguably violate the implicit terms of access.[2]

3.1.2 Data Integrity and Falsification

The Proclamation also prohibits the alteration or falsification of computer data. When storing news articles in the local MongoDB instance, the system must ensure **Read-Only Integrity**. The content scraped must be an exact replica of the source. Any modification—such as summarizing or rewriting headlines—must be clearly labeled as a derivative work to avoid

accusations of spreading misinformation or "false computer data," which carries severe penalties.[2]

3.2 Global Scraping Ethics and the Robots Protocol

Beyond Ethiopian law, the system must adhere to global best practices to ensure longevity and ethical operation.[11]

3.2.1 Robots.txt Compliance

The robots.txt file is the standard mechanism for websites to communicate with crawlers. Before any scraping session begins, the system must fetch https://addisfortune.news/robots.txt and https://capitalethiopia.com/robots.txt. The parsing logic must respect Disallow directives. If a specific path (e.g., /admin/ or /private-reports/) is disallowed, the scraper must strictly avoid it.

3.2.2 User-Agent Identification

Hiding behind a fake User-Agent (spoofing) is necessary for *Reuters* due to their aggressive blocking, but for local sites, transparency is preferred. For *Capital* and *Fortune*, the scraper should identify itself with a custom User-Agent string such as EthiopianFinancialIntelBot/1.0 (+https://your-domain.com/bot-policy). This allows the webmasters to contact the operator if the bot is causing issues, rather than simply blocking the IP address.[12]

---

4. Architectural Design: The Clean Architecture Approach

To manage the complexity of scraping multiple sources with different technical requirements and storing them in a unified database, we adopt the **Clean Architecture** pattern (also known as the Onion Architecture). This design philosophy emphasizes the separation of concerns, ensuring that the "business logic" (extracting financial insights) is independent of the "infrastructure" (the web frameworks and databases).[13]

4.1 Architectural Layers

4.1.1 The Domain Layer (Core)

At the center of the architecture is the **Domain Layer**. This layer defines the fundamental entities of our system. Here, we define what a NewsArticle is. It includes attributes like title, content, published_date, source, url, and keywords. This layer has *zero dependencies* on external libraries like FastAPI or Scrapy. It is pure Python code. This ensures that if we switch from MongoDB to PostgreSQL in the future, our core business logic remains untouched.

4.1.2 The Use Case Layer (Application)

Surrounding the domain is the **Use Case Layer**. This layer contains the specific application logic.

- `HarvestNewsUseCase`: Orchestrates the fetching of news from different sources.
- `FilterNewsUseCase`: Applies the keyword taxonomy (Treasury Bills, Stocks) to incoming raw text.
- `GetNewsUseCase`: Handles the logic for the API endpoint, including pagination and search filtering.

4.1.3 The Interface Adapter Layer

This layer adapts data from the outside world to the format most convenient for the Use Cases.

- **Scraper Adapters:** We will implement a `BaseScraper` interface. Specific implementations (`ReutersScraper`, `CapitalScraper`, `FortuneScraper`) will plug into this interface. This allows the application to treat all sources uniformly, despite their vastly different internal workings (Playwright vs. HTTPX).
- **Repository Adapters:** This layer implements the database logic. The `MongoRepository` will handle the specifics of connecting to MongoDB and executing queries using `Beanie`.

4.1.4 The Infrastructure Layer

The outermost layer contains the frameworks and drivers.

- **Web Framework: FastAPI** is selected for its high performance, native async support, and automatic OpenAPI documentation.[15]
- **Database: MongoDB** is chosen for its schema flexibility, essential for dealing with unstructured news data.
- **Browser Automation: Playwright** resides here as the engine driving the Reuters scraper.

4.2 Asynchronous Concurrency Model

A critical architectural decision is the use of **Asynchronous I/O**. Web scraping is an "I/O bound" task, meaning the CPU spends most of its time waiting for network responses.

- **Synchronous Approach:** The scraper requests a page from *Capital*, waits 2 seconds for the server, processes it, then requests the next. This is inefficient.
- **Asynchronous Approach:** The scraper fires a request to *Capital*, and while waiting for the response, it fires a request to *Fortune* and another to *Reuters*. Python's `asyncio` library allows a single thread to manage thousands of concurrent connections.[16]

We will utilize **HTTPX** for async HTTP requests and **Motor** (the async driver for MongoDB) to ensure the entire pipeline is non-blocking. This allows the API to serve user requests even while the scraper is actively harvesting thousands of pages in the background.

---

5. The Scraping Engine Implementation

This section details the engineering of the "Harvester" module, the component responsible for the actual retrieval of data.

5.1 The Abstract Scraper Interface

To enforce consistency, we define an abstract base class.

Python

```
None


from abc import ABC, abstractmethod
from typing import List
from src.domain.entities import Article

class BaseScraper(ABC):
    @abstractmethod
    async def connect(self):
        """Establish session or browser context."""
        pass

    @abstractmethod
    async def fetch_latest_articles(self, limit: int = 10) ->
List[Article]:
        """Retrieve the most recent articles."""
        pass

    @abstractmethod
    async def parse(self, raw_html: str) -> Article:
        """Convert HTML to Domain Entity."""
        pass
```

5.2 The Static Scraper: Capital and Fortune

For the Ethiopian sites, efficiency is paramount. We use `HTTPX` combined with `BeautifulSoup` using the `lxml` parser.

5.2.1 High-Performance Parsing with LXML

While Python's built-in HTML parser is available, `lxml` is orders of magnitude faster because it is implemented in C.[17] When scraping thousands of historical archives from *Capital Ethiopia*, this parsing speed difference becomes significant.

Capital Ethiopia Logic:

The scraper targets the article tag.

- **Title:** Extracted from `h2.entry-title`.
- **Date:** Extracted from `time.entry-date published`.
- **Body:** Extracted from `div.entry-content`.
- **Link:** Extracted from the `a` tag inside the title.

Error Handling: The scraper implements a "Circuit Breaker". If *Capital* returns 5 consecutive errors (e.g., 500 Internal Server Error), the scraper temporarily disables that source for 1 hour to prevent hammering a down server.

5.3 The Dynamic Scraper: Reuters and Cloudflare Bypass

Scraping *Reuters* requires overcoming Cloudflare. We utilize **Playwright** with the **Stealth** plugin.

5.3.1 Playwright vs. Selenium

We choose Playwright over Selenium because:

1. **Speed:** Playwright interacts directly with the browser protocol (CDP), making it faster than Selenium's WebDriver.
2. **Async Native:** Playwright was built for async/await, integrating seamlessly with our FastAPI stack.
3. **Stealth:** The `playwright-stealth` library is highly effective at masking the automation indicators (e.g., `navigator.webdriver` flag) that Cloudflare looks for.[7]

5.3.2 Implementation Strategy

The `ReutersScraper` class manages a browser context.

1. **Initialization:** Launches a Chromium instance in headless mode, injecting stealth scripts.
2. **Navigation:** Navigates to `https://reuters.com/world/africa`.
3. **Waiting:** Uses `page.wait_for_selector()` rather than `time.sleep()`. This waits for specific React components (like the news feed list) to render.

4. **Extraction:** Executes JavaScript in the browser context (`page.evaluate()`) to extract text. This is more reliable than parsing the HTML because React can change the DOM structure dynamically.

Code Snippet: Cloudflare Bypass Context

Python

```
None


async with async_playwright() as p:
    browser = await p.chromium.launch(headless=True,
args=["--disable-blink-features=AutomationControlled"])
    context = await browser.new_context(
        user_agent="Mozilla/5.0 (Windows NT 10.0; Win64; x64)
AppleWebKit/537.36..."
    )
    # Inject stealth scripts to override navigator properties
    await context.add_init_script("Object.defineProperty(navigator,
'webdriver', {get: () => undefined})")
    page = await context.new_page()
    await page.goto("https://reuters.com/world/africa")
```

This specific script overrides the `navigator.webdriver` property, which is the primary "tell" for Cloudflare that a browser is being controlled by automation.

---

6. Data Engineering and NLP: The Knowledge Filter

The user requirement is specific: filter for "capital markets," "treasury bills," "macroeconomics," etc. Raw scraping yields noise (sports, lifestyle). The NLP layer is the filter.

6.1 Algorithmic Filtering: FlashText vs. Regex

Standard approaches use Regular Expressions (Regex) to find keywords. However, Regex performance degrades linearly with the number of keywords. If we have 500 financial terms, Regex scans the text 500 times (or creates a massive, slow compiled object).

We will use the **FlashText** algorithm (Aho-Corasick implementation).

- **Performance:** FlashText scans the document exactly *once*, regardless of whether we are searching for 5 keywords or 5,000. It builds a Trie (prefix tree) structure.
- **Benchmark:** In tests involving large financial corpuses, FlashText has shown to be up to 100x faster than Regex.[18]

6.2 The Financial Taxonomy

We define a rigorous taxonomy for the Ethiopian market context.

- **Category: Treasury Bills**
    - Keywords: "T-Bill", "Treasury Bill", "28-day bill", "91-day bill", "Yield", "Auction", "National Bank of Ethiopia".
- **Category: Capital Markets**
    - Keywords: "ESX", "Ethiopian Securities Exchange", "IPO", "Initial Public Offering", "Shareholder", "Dividend", "Equity", "Brook Taye" (ESX CEO).
- **Category: Macroeconomics**
    - Keywords: "Inflation", "CPI", "GDP", "Forex", "Exchange Rate", "Birr", "IMF", "World Bank", "Liberalization".
- **Category: Politics (Relevant to Market)**
    - Keywords: "Proclamation", "Parliament", "Abiy Ahmed", "Ministry of Finance", "Regulation".

The NLP processor passes every scraped article through this taxonomy. Only articles matching at least one keyword are tagged and stored. If an article matches multiple (e.g., "Parliament discusses ESX proclamation"), it receives multiple tags, enriching the search capability of the API.

---

7. Persistence Layer: MongoDB and Beanie

The database choice is dictated by the data's nature. News is semi-structured. Headlines always exist, but "sub-headlines," "authors," and "image captions" vary.

7.1 Schema Design with Beanie

Beanie is an ODM that bridges Pydantic and MongoDB. It allows us to define schemas as Python classes, providing type safety and validation.[19]

Python

None

```
from beanie import Document, Indexed
from pydantic import Field
from datetime import datetime
from typing import List, Optional

class NewsArticle(Document):
    title: str
    # Indexed unique URL prevents duplicate entries if scraped twice
    url: Indexed(str, unique=True)
    source: str
    published_date: datetime
    # Text index allows full-text search in MongoDB
    content: str
    summary: Optional[str] = None
    tags: List[str] =

    class Settings:
        name = "articles"
        indexes = [("title", "text"), ("content", "text")], #
Compound Text Index
            "published_date",
            "tags"
        ]
```

7.2 Indexing Strategy for Performance

To support the user's "Get News" endpoint efficiently:

1. **Compound Text Index:** We create an index on `title` and `content`. This allows MongoDB to perform high-speed text searches (e.g., "Find articles mentioning 'cement'") without scanning every document.[21]
2. **Date Index:** Financial queries are almost always time-bound (e.g., "News from last week"). Indexing `published_date` is mandatory for query performance.
3. **Tag Index:** Allows instant retrieval of all "Treasury Bill" articles.

---

8. API Layer: FastAPI Implementation

The API is the delivery mechanism. It exposes the rich data stored in MongoDB to the user.

8.1 Dependency Injection and Configuration

We use `pydantic-settings` to manage configuration (Database URL, API Keys) via environment variables. This keeps secrets out of the codebase.[22] FastAPI's dependency injection system manages the database connection lifecycle, ensuring that connections are pooled and reused efficiently.

8.2 The "Get News" Endpoint

The endpoint `GET /api/v1/news` is designed to be a flexible query interface.

Parameters:

- `q` (Optional string): Free text search.
- `category` (Optional string): Filter by the taxonomy tags (e.g., "Macroeconomics").
- `source` (Optional string): Filter by provider (e.g., "Reuters").
- `start_date` / `end_date`: For time-series analysis.
- `limit` / `skip`: For pagination.

Implementation Logic:

Inside the route handler, we construct a dynamic MongoDB query based on the parameters provided.

Python

```
None


search_query = {}
if category:
    search_query["tags"] = category
if source:
    search_query["source"] = source
if start_date:
    search_query["published_date"] = {"$gte": start_date}

# If 'q' is present, we use MongoDB's text search operator
if q:
```

```
    search_query["$text"] = {"$search": q}

articles = await
NewsArticle.find(search_query).sort("-published_date").limit(limit).
to_list()
```

This logic ensures that the API is not just a dump of data, but a query engine capable of answering specific questions like "Show me all Treasury Bill news from Capital Ethiopia published in January 2025."

---

9. Deployment and Operational Strategy

To move from code to a production system, we employ **Docker** and **Continuous Integration**.

9.1 Docker Containerization

We define a `docker-compose.yml` that orchestrates the services:

1. **FastAPI Service:** Runs the API and the Scraper scheduler.
2. **MongoDB Service:** Local database instance (persisting data to a Docker volume).
3. **Mongo Express:** A lightweight web UI for inspecting the database manually.

9.2 Task Scheduling

We use **APScheduler** (Advanced Python Scheduler) running inside the FastAPI process.

- **Job 1:** "Quick Harvest" - Runs every 30 minutes. Checks the homepage of all three sources for new articles.
- **Job 2:** "Deep Archive" - Runs once a week. Traverses historical pages to backfill data.

9.3 Monitoring

We integrate structured logging using the `loguru` library. Logs track:

- Scraping success/failure rates per source.
- Number of new articles added.
- API request latency.
  This observability is crucial for identifying when a source changes its HTML structure or when Cloudflare defenses block the scraper.

---

10. Conclusion

This report has detailed the architecture for a sovereign Ethiopian Financial Intelligence System. By synthesizing the speed of static scraping for local sources with the sophistication of headless automation for international wires, and underpinning it with a legally compliant, keyword-driven architecture, the system is positioned to provide the "exhaustive" and "nuanced" market view requested. The combination of **FastAPI**, **Beanie**, **FlashText**, and **Playwright** represents the current state-of-the-art in Python data engineering, ensuring that as the Ethiopian capital market grows, this infrastructure can scale alongside it.

The immediate next steps involve the development of the "Harvester" prototypes and the validation of the Cloudflare bypass mechanism, which remains the highest technical risk. Once operational, this system will transform raw web noise into actionable financial signal.